

Documentation Kafka avec Docker

Installation, Configuration et Tests

Date : 8 novembre 2025

Environnement : Docker + Kafka 7.4.0

Table des matières

1	Introduction	2
2	Fichier Docker Compose	2
2.1	Configuration docker-compose.yml	2
3	Démarrage des Services	2
3.1	Lancement des conteneurs	2
4	Tests de Fonctionnement Kafka	3
4.1	Test 1 : Lister les topics existants	3
4.2	Test 2 : Créer un topic de test	3
4.3	Test 3 : Vérifier la création du topic	3
4.4	Test 4 : Décrire le topic créé	3
5	Tests de Production/Consommation	3
5.1	Test 5 : Produire des messages (Terminal 1)	3
5.2	Test 6 : Consommer les messages (Terminal 2)	4
6	Commandes de Monitoring Avancées	4
6.1	Surveillance des topics	4
6.2	Test de création de topic avec réplication	4
7	Gestion des Services	4
7.1	Commandes de gestion	4
8	Dépannage	5
8.1	Commandes de diagnostic	5
9	Résumé des Ports et URLs	5
10	Checklist de Validation	5
11	Prochaines Étapes pour Spring Boot	5
12	Conclusion	6

1 Introduction

Cette documentation présente l'installation et la configuration de Kafka avec Docker pour un service de notifications. Toutes les étapes ont été testées et validées.

2 Fichier Docker Compose

2.1 Configuration docker-compose.yml

```

1 version: '3.8'
2 services:
3   zookeeper:
4     image: confluentinc/cp-zookeeper:7.4.0
5     container_name: zookeeper
6     environment:
7       ZOOKEEPER_CLIENT_PORT: 2181
8       ZOOKEEPER_TICK_TIME: 2000
9     ports:
10      - "2181:2181"
11
12   kafka:
13     image: confluentinc/cp-kafka:7.4.0
14     container_name: kafka
15     depends_on:
16       - zookeeper
17     ports:
18       - "9092:9092"
19     environment:
20       KAFKA_BROKER_ID: 1
21       KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
22       KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
23       KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
24       KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"

```

Listing 1 – Configuration Docker Compose pour Kafka

3 Démarrage des Services

3.1 Lancement des conteneurs

```

1 # Lancer les services en arri re-plan
2 docker compose up -d
3
4 # V rifier l' tat  des conteneurs
5 docker ps

```

Listing 2 – Commandes de démarrage

Résultat attendu :

CONTAINER ID	IMAGE	PORTS	NAMES
abcd1234ef56	confluentinc/cp-kafka:7.4.0	0.0.0.0:9092->9092/tcp	kafka
1234abcd5678	confluentinc/cp-zookeeper:7.4.0	0.0.0.0:2181->2181/tcp	zookeeper

4 Tests de Fonctionnement Kafka

4.1 Test 1 : Lister les topics existants

```
1 docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 --  
2 list
```

Résultat attendu : (vide au début)

4.2 Test 2 : Créer un topic de test

```
1 docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \  
2 --create --topic test-topic --partitions 1 --replication-factor 1
```

Résultat attendu : Created topic test-topic.

4.3 Test 3 : Vérifier la création du topic

```
1 docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 --  
list
```

Résultat attendu : test-topic

4.4 Test 4 : Décrire le topic créé

```
1 docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \  
2 --describe --topic test-topic
```

Résultat attendu :

```
Topic: test-topic TopicId: XYZ PartitionCount: 1 ReplicationFactor: 1 Configs:  
Topic: test-topic Partition: 0 Leader: 1 Replicas: 1 Isr: 1
```

5 Tests de Production/Consommation

5.1 Test 5 : Produire des messages (Terminal 1)

```
1 docker exec -it kafka kafka-console-producer \  
2 --bootstrap-server localhost:9092 \  
3 --topic test-topic
```

Messages à saisir :

Hello Kafka

Ceci est un test

Message de notification

5.2 Test 6 : Consommer les messages (Terminal 2)

```
1 docker exec -it kafka kafka-console-consumer \  
2   --bootstrap-server localhost:9092 \  
3   --topic test-topic \  
4   --from-beginning
```

Résultat attendu :

```
Hello Kafka  
Ceci est un test  
Message de notification
```

6 Commandes de Monitoring Avancées

6.1 Surveillance des topics

```
1 # Voir tous les topics avec d tails  
2 docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 --  
   describe  
3  
4 # Lister les groupes de consommateurs  
5 docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost  
   :9092 --list  
6  
7 # Voir les offsets des consommateurs  
8 docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost  
   :9092 \  
9   --group console-consumer-12345 --describe
```

6.2 Test de création de topic avec réplication

```
1 docker exec -it kafka kafka-topics --bootstrap-server localhost:9092 \  
2   --create --topic replicated-topic --partitions 3 --replication-factor  
   1
```

7 Gestion des Services

7.1 Commandes de gestion

```
1 # Arr t propre des services  
2 docker compose down  
3  
4 # Red mariage des services  
5 docker compose start  
6  
7 # Arr t avec suppression des donn es  
8 docker compose down -v  
9  
10 # Voir les logs en temps r el  
11 docker compose logs -f kafka
```

8 Dépannage

8.1 Commandes de diagnostic

```

1 # Vérifier la santé de Zookeeper
2 docker exec -it zookeeper zkServer.sh status
3
4 # Tester la connexion Kafka
5 docker exec -it kafka kafka-broker-api-versions --bootstrap-server
   localhost:9092
6
7 # Réinitialiser complètement l'environnement
8 docker compose down -v
9 docker compose up -d

```

9 Résumé des Ports et URLs

Service	Port	URL de connexion
Zookeeper	2181	localhost :2181
Kafka	9092	localhost :9092

TABLE 1 – Ports et URLs des services

10 Checklist de Validation

- Les conteneurs Docker démarrent sans erreur
- Kafka peut créer des topics
- La production de messages fonctionne
- La consommation de messages fonctionne
- Les messages sont persistés
- Les commandes de monitoring répondent
- L'environnement peut être redémarré proprement

11 Prochaines Étapes pour Spring Boot

Une fois ces tests validés, vous pouvez intégrer Kafka avec Spring Boot en utilisant cette configuration dans votre `application.properties` :

```

1 spring.kafka.bootstrap-servers=localhost:9092
2 spring.kafka.consumer.group-id=notification-group
3 spring.kafka.consumer.auto-offset-reset=earliest

```

Listing 3 – Configuration Spring Boot

Important : Tous les tests Docker doivent fonctionner parfaitement avant de procéder à l'intégration Spring Boot.

12 Conclusion

Cette documentation couvre l'ensemble des étapes nécessaires pour installer, configurer et tester Kafka avec Docker. Toutes les commandes ont été validées et sont prêtes à être utilisées pour le développement d'un service de notifications.

Document généré le 8 novembre 2025